O7 Solutions

Bus Ticket Booking Backend

1. **Use of any GPT or any AI chatbot is not allowed; if found, you will be disqualified from the test**
2. **Test Duration is 1:30 hours**
3. **All Parts are valuable in performance**
4. **Maintain a Postman Collection for Test or Evaluation**
5. **You can use MongoDB or MySQL as a DB**

Section 1: Authentication

Users can sign up as **passengers** or **admins**.

- ❖ POST /auth/register → { name, email, password, role }
- ❖ POST /auth/login → Return JWT (expiry 1h)
- ❖ Middleware authMiddleware to protect all routes except /auth/*
- ❖ req.user = { id, role }
- ❖ Passwords hashed using **bcrypt**

Section 2 : Booking Management (CRUD & Calculations)

- ❖ **POST /bookings** → Create booking and calculate fare using rules:
- ❖ **GET /bookings?page=&limit** → List bookings with pagination
- ❖ **GET /bookings/:id** → Get booking details
- ❖ **PUT /bookings/:id** → Update booking (recalculate fare if inputs change)
- ❖ **DELETE /bookings/:id** → Delete booking

Section 3: Reporting

- **GET /reports/class/:travelClass** → Total bookings and total fare for a specific class (e.g., Luxury)
- **GET /reports/fare-over/:amount** → List bookings where finalFare > amount
- **GET /reports/daily-summary** → For each day:
  - Total bookings
  - Total revenue
  - Average fare per booking

Section 4: Feature

GET /bookings/history/:userId?page=&limit= → Paginated history of bookings per passenger

Include filters:

- TravelClass = Luxury
- FinalFare > 1000

HINT: Booking Schema

```
JS test.js
1   {
2     passengerName: String,
3     age: Number,
4     distance: Number,
5     tickets: Number,
6     travelClass: String, // Normal / AC / Sleeper / Luxury
7     travelTime: String, // Peak / Off-Peak
8     fareBreakdown: {
9       baseFare: Number,
10      peakSurcharge: Number,
11      ageDiscount: Number,
12      groupDiscount: Number,
13      serviceFee: Number,
14      GST: Number,
15      finalFare: Number
16    },
17    createdAt: Date
18  }
19
```